

AI Course

Capstone Project Final Report

For students (instructor review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Bayan: On-Device Arabic Text Simplification for Readers with Dyslexia

تبسيط · Simplify

30/09/26

Cogni

Marwan Elamami
Abdulrahman Khengari
Ahmed Alaeb
Sanad Ali
Mohammed Thabet
Abdul Majid Mraied

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Related Work
- 1.4. Members and Role Assignments
- 1.5. Schedule and Milestones

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Design
- 2.5. BayanBench

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. On the Device
- 3.5. User Interface
- 3.6. Testing and Improvements

4. Projected Impact

- 4.1. Accomplishments and Benefits
- 4.2. Future Improvements
- 4.3. Limitations
- 4.4. Conclusion
- References

5. Team Member Review and Comment

6. Instructor Review and Comment

Abstract. Arabic poses problems for readers with dyslexia that tools built for Latin scripts do not address: short vowels go unwritten, clitics fuse into long words, and clauses chain across whole paragraphs. We present **Bayan**, an Android system that rewrites a selection of Arabic text, from inside any application and entirely on the phone, into a form that is easier to decode. We first show that the standard evaluation misleads for this task: on SAMER, the only human-written Arabic simplification corpus, copying the input scores 77.5 SARI, and a fine-tuned AraT5v2 that beats copying on the sentences a human changed still loses to it overall. We then train two encoder–decoder models, AraT5v2 (368M parameters) and AraBART (139M), on a 22,733-pair corpus in which every unchanged pair is kept, LLM-written pairs enter only through a meaning filter derived from human rewrites, and a strength tag conditions each source. Building that corpus exposed that 9.3% of a public Arabic simplification dataset overlaps the standard test sets. To measure what a reader needs rather than agreement with one editor, we introduce **BayanBench**, 1,981 items in 13 behavioural tracks and four item sets, split by document and scored per measure with cluster-bootstrap intervals. On its test split, the medium strength setting removes every clause over 15 words in 50.8–55.1% of long-clause items and reduces hard words in 41.2–44.0% of items, while keeping at least 60% of the source’s words in 81.7–88.2%. It also over-edits easy text, rewrites protected scripture, and completes cut-off selections, failures that a reference-based score does not surface. Quantised to int8, the AraBART bundle takes 222 MB; the larger AraT5v2 bundle, run on a Snapdragon 870 phone, shows the first word of a rewrite 177–318 ms after the request, and of each later sentence within 85 ms.

1. Introduction

1.1. Background Information

Developmental dyslexia affects an estimated 11% of Arab primary-school children, pooled across 18 studies ($N = 30,243$) [1]. It impairs reading through phonological processing, and Modern Standard Arabic adds obstacles of its own. Short vowels are normally left unwritten, so one written form carries several readings: **كتب** is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books), and the reader must resolve it from context. Clitics attach directly to the stem, producing long words that resist chunking, and Arabic-speaking children with reading disabilities show early deficits in exactly this morphological layer [2]. Prose chains clauses with **و** and **ف** across whole paragraphs, so one sentence can exceed working memory.

The evidence on what helps is uneven, and we build on the parts that have been measured. More frequent and shorter words sped reading and improved comprehension for readers with dyslexia in eye-tracking studies [3, 4]. Unpacking bound morphemes into separate words helped Arabic readers with reading disabilities and not their typical peers, the one Arabic finding with a tested group interaction [5]. Texts simplified by rules that split subordinate and coordinated clauses into short sentences were read faster and understood better by beginning readers, and most by the weakest of them [6]. Full diacritization, by contrast, is contested: it adds visual crowding [7], and its benefit for our readers has not been tested. We therefore treat *simplify first* as the core intervention and diacritization as an optional later stage.

1.2. Motivation and Objective

Readers meet difficult Arabic inside other applications — a message, a news page, a PDF — not in a reading app. Bayan therefore appears where the text already is: selecting text on an Android phone offers **تبسيط** (*Simplify*) in the system selection menu, and the rewrite opens in

a panel over the same app. Every model runs on the phone, so the reader’s text never leaves it and the tool works offline.

The closest prior systems leave this setting open. Dyslexia-targeted neural simplification exists for English [8]; Arabic simplification corpora and models exist without a dyslexia target [10, 12]; and the one Arabic dyslexia study with a language model used it as a tutor, not as an automatic rewriter [9]. Our literature searches of 17 and 23 September found no system that combines the three: Arabic, a dyslexia-oriented rewrite, and on-device inference. Building one raises three questions, which organise this report:

- RQ1** Does the standard reference-based evaluation measure what this task needs, and what does a model learn from the only human-written corpus?
- RQ2** Can one model, trained on meaning-filtered mixed-domain data with a strength tag, make Arabic text measurably easier to decode without losing its content?
- RQ3** Does it run on a mid-range phone at interactive latency, and what do quantisation and the deployed input format cost?

We set four objectives against which subsection 4.1 reports the outcome:

- O1** A simplifier that beats copying the input on held-out sentences a human simplified, and that makes text measurably easier without losing its meaning.
- O2** Diacritics restored on the simplified text without altering its letters.
- O3** The pipeline running on a mid-range Android phone, within 250 MB for the simplifier.
- O4** Evidence from readers with dyslexia, collected with informed consent.

Contributions.

- **An evaluation audit.** We show that copy-heavy test sets make SARI reward inaction, report every score beside a copy baseline and split by whether a human changed the sentence, and document that 9.3% of a public Arabic simplification corpus overlaps the SAMER and BAREC test sets (subsection 3.1, subsection 3.3).
- **A training recipe for one model at several strengths,** built from that audit: all unchanged pairs kept, LLM-written pairs admitted by a meaning filter calibrated on human rewrites, and a strength tag that becomes the reader’s setting (subsection 2.2).
- **BayanBench,** a behavioural benchmark for dyslexia-oriented Arabic rewriting with deterministic and judge tiers, document-level splits, cluster-bootstrap intervals, and a judge-acceptance protocol fixed before any judge was tested (subsection 2.5).
- **An on-device system,** with int8 encoder–decoder bundles decoded token by token inside an Android app and measured on a phone (subsection 2.4, subsection 3.4).

subsection 1.3 places the work in the literature. Section 2 describes the data, the models, the system and the benchmark; Section 3 reports the results for each research question; Section 4 draws the conclusions and states the limitations.

1.3. Related Work

Arabic simplification data. SAMER [10] is the only human-written multi-level Arabic simplification corpus: 15 novels rewritten at two easier levels, each version keeping the original’s sentences by design. BAREC [11] labels 69,441 sentences with 19 human reading levels but is

not parallel. Baseet [12] provides three LLM-written levels per source, and DAASI [13] human-verified rewrites of government and insurance text. No corpus targets readers with dyslexia, and none teaches sentence splitting: a sentence is added in 0% of SAMER pairs, 2–4% of Baseet’s and 8% of DAASI’s (subsection 3.2).

Controllable simplification. A control token prepended to the source lets one model rewrite for different audiences, first with target-grade tokens [15] and later with measured attributes such as length and lexical complexity [16, 17]. Our strength tags follow this line with one difference: each tag names the corpus style it was learned from, so that a meaning filter can be applied to exactly the sources that need one.

Evaluating simplification. SARI [18] is the field’s standard, but it correlates only $r = 0.36$ with human judgements of simplicity [20], and similarity-based metrics detect deleted content but not inserted or substituted content [21]. Reference-free estimates of the reading level track simplicity better [22], and edit-level error taxonomies track meaning [23]. For benchmark design we follow behavioural testing [24], construct-validity and reporting checklists [25, 26], and document-clustered intervals [27]. LLM judges are known to favour outputs close to data they wrote or approved [28], which shapes how we restrict ours.

1.4. Members and Role Assignments

Every member merged work into the team repository; the evidence column cites the pull requests and issues that hold it (#).

Table 1. Members, roles, and the work each delivered.

Member	Role	Delivered (evidence)
Marwan Elamami	Team lead; integration; on-device model	Product-form decision (#9); evaluation protocol and fixes (#16, #18); model 1 diagnosis; Baseet acquisition and leakage filtering; model-2 recipe, data and task specifications (#27–#34); on-device engine and app; phone test; BayanBench; Action Plan (#25); this report
Abdulrahman Khengari	Data lead	Validators for generated pairs (#10, #11); CAMEL readability check (#17, #20); synthetic corpus of 13,072 pairs, split and published (#8, #38, #39)
Ahmed Alaeb	Model training	SAMER conversion; model 1 trained and documented (#5, #22); model 2 on AraT5v2 trained and published (#28, #40)
Sanad Ali	Diacritization; co-training	Four-system diacritization benchmark; Libtashkeel selected (#6, #23); model 2 on AraBART and the speed benchmark (#29, #35)
Mohammed Thabet	Android application	PROCESS_TEXT plugin, result sheet and 41 tests (#14, #21); model download pipeline, sheet states and typography settings (#31, #37)
Abdul Majid Mraied	Evaluation	Scoring and leakage scripts (#7, #15); locked-set converter, copy baselines, generate-and-score path (#19, #24, #26)

1.5. Schedule and Milestones

The work reported here ran from 13 to 28 September 2026, ahead of submission on 5 October and the pitch on 6 October. It went through two complete build-and-measure cycles. Model 1 was trained, scored on locked test sets and diagnosed. Model 2 was designed from that diagnosis, trained on two architectures, exported to the phone, tested on a device, and scored on a benchmark built for the task. Figure 1 shows the timeline; Table 2 compares target and actual dates.

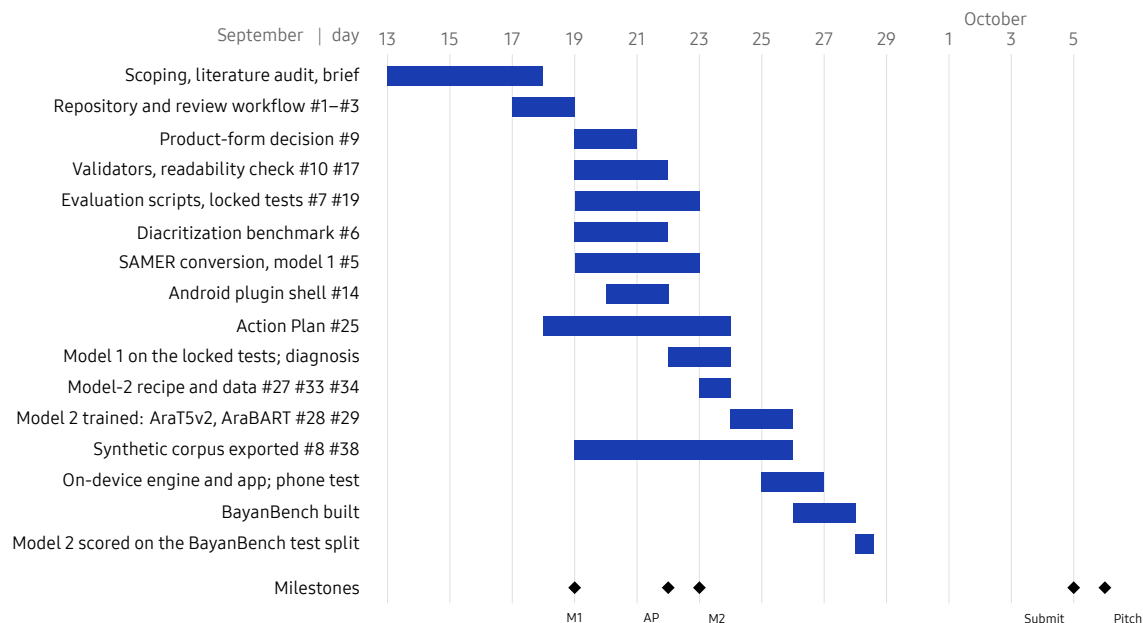


Figure 1. Timeline as executed, 13–28 September. AP = Action Plan; M1, M2 = mentoring sessions. Numbers after a task are its GitHub issues.

Table 2. Milestones: target against actual.

Milestone	Target	Actual
Evaluation scripts ready (#7)	Mon 21 Sep	Mon 21 Sep
Diacritizer selected (#6)	Mon 21 Sep	Mon 21 Sep
Android plugin shell (#14)	—	Mon 21 Sep
Action Plan submitted	Tue 22 Sep	Tue 22 Sep
Model 1 scored on the locked test sets	Wed 23 Sep	Wed 23 Sep
Model-2 data built; tasks issued (#28–#32)	—	Thu 24 Sep
Model 2 trained, both architectures (#28, #29)	Fri 25 Sep	Fri 25 Sep
Synthetic corpus exported (#8, #38)	Fri 25 Sep	Sat 26 Sep
Model running on a phone, latency measured	Fri 2 Oct	Sat 26 Sep
BayanBench built; model 2 scored on its test split	—	Mon 28 Sep

2. Project Execution

2.1. Data Acquisition

We acquired five corpora and produced a sixth (Table 3). SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed, and no SAMER sentence is reproduced in this report. Baseeet is used with its authors’ permission. Two test splits, SAMER test and BAREC test, were locked before any training.

The synthetic corpus. Our generation pipeline takes BAREC train and dev sources, draws five candidates per source from DeepSeek V4.1 Flash, and accepts one only if an LLM equivalence validator and the CAMEL BAREC readability model both pass it. The validator is a DSPy program optimised with MIPROv2 and reaches 84% accuracy on 25 held-out annotated pairs at the chosen threshold. The export is split by a salted hash of the normalised source, so that a sentence

Table 3. Corpora. Sizes are rows in the files we hold.

Corpus	Size	Licence	Used for
SAMER [10]	14,343 / 2,983 / 3,277 pairs (train / dev / test)	research; written approval	models 1 and 2; locked test set
BAREC [11]	54,845 / 7,310 / 7,286 sentences	CC BY-SA 4.0	locked test set; BayanBench core items; generation sources
Baseet [12]	34,294 sources, three levels each; 31,104 after filtering	with permission	model 2 (6,888 meaning-filtered pairs)
DAASI [13]	1,509 pairs + 350 held out	CC0	model 2 (1,502 pairs); out-of-domain test
Tashkeela [14]	743 test sentences	open	diacritization benchmark
Bayan synthetic v0	13,072 pairs (12,419 generated, 563 identity, 90 scripture)	CC BY-SA 4.0, published	a released artefact; not used to train model 2

repeated under several BAREC identifiers never straddles two splits; a classifier trained to tell the splits apart scores AUC 0.51. BAREC test was never used. The corpus was completed after model 2 had been trained, so no result in this report depends on it.

2.2. Training Methodology

Task. Given a source sentence x and a strength tag t , the model produces a rewrite $\hat{y} = f(t \oplus x)$ that should keep the meaning of x and be easier to decode. Training minimises token-level cross-entropy on pairs $(t \oplus x, y)$; decoding is greedy, the setting the phone runs.

Base models. Simplification is a sequence-to-sequence task, so every candidate is a pre-trained encoder–decoder transformer (Table 4). AraT5v2 came first because it beat AraBART at all three levels in Baseet’s head-to-head evaluation; AraBART is the on-device candidate because it is 2.6 times smaller.

Input normalisation. No vocabulary in Table 4 was built for diacritized text: in AraT5v2 only 1.6% of pieces carry a diacritic, and AraBART cannot represent one at all. Every model therefore receives its source with tashkeel and tatweel removed, in training and in the app; subsection 3.3 measures what happens otherwise.

Model 1. We fine-tuned AraT5v2 to rewrite SAMER’s level-5 sentences into their level-3 simplifications with the prefix **بسط** (*simplify*): AdamW at learning rate 10^{-4} with weight decay 0.01 and 100 warm-up steps, an effective batch of 16, mixed precision, 10 epochs (3,250 steps) and seed 42. Pairs whose simplification equals the source (42% of SAMER train) were capped at 20% of training, and the checkpoint with the best SARI on a 500-row dev subset was kept.

Model 2. The diagnosis of model 1 (subsection 3.3) identified three problems, and the model-2 recipe changes one thing for each:

- Inaction was rewarded for the wrong reason.* Capping unchanged pairs at 20% while choosing the checkpoint on a dev sample that is 43% unchanged pulls the model both ways. Model 2 keeps all 6,033 unchanged SAMER pairs, so it learns when not to edit, and selects its checkpoint on SARI over dev rows a human changed.
- SAMER is novels, and it teaches word substitution.* Model 2 adds DAASI’s everyday and administrative text and Baseet’s educational text. Baseet’s targets are LLM-written, and many are summaries that drop facts, delete answer options or invent a speaker. A Baseet pair is

Table 4. Candidate base models. “Tashkeel” is measured on 1,000 diacritized BAREC dev sentences: vocabulary entries containing a diacritic, and how many more tokens the same sentences cost with diacritics than without.

	AraT5v2-base-1024	AraBART	HPLT T5 base (Arabic)
Family	T5 [32]: span-corruption pretraining, relative position buckets	BART [33]: denoising pretraining, learned positions	T5 in the NorT5 variant [34]
Layers (enc/dec), width	12/12, $d=768$, 12 heads, gated-GELU FFN 2048	6/6, $d=768$, 12 heads, GELU FFN 3072	12/12, $d=768$, 12 heads, FFN 2048
Parameters	368M (46% in two untied $110k \times 768$ embeddings)	139M	$\approx 294M$
Vocabulary	110,208 SentencePiece	50,002 SentencePiece BPE	32,768
Context	1,024 tokens	1,024 positions	512 positions
Pretraining	Arabic MSA and dialects [29, 30]	large Arabic corpora [31]	HPLT v3 web crawl, Arabic subset [35]
Tashkeel	1.6% of pieces; $2.57 \times$ tokens	0 pieces; every mark becomes <unk>; $4.53 \times$ tokens	0 pieces; $1.81 \times$ tokens
Licence	none stated on the model card	Apache-2.0	Apache-2.0
Used in	models 1 and 2	model 2	not trained

kept only if it passes a *meaning filter*: LaBSE similarity ≥ 0.646 , a target at least 67% of the source’s length, and every number kept. Each threshold is the 5th percentile of the same measure over DAASI’s human-verified rewrites, so a pair is rejected only when it departs from its source more than 95% of human simplifications do.

- (c) *One strength does not fit every reader.* Every source carries a tag naming the style it was learned from (Table 5). At run time the tag is the reader’s strength setting, so one model serves light, medium and strong rewrites [15, 16].

Table 5. Model-2 training set (22,733 pairs). For each Baseet source, the mildest level whose target is shorter per sentence or has fewer rare or long words is used. No source overlaps any test set.

Tag	Taught by	Pairs	Meaning at run time
[S0]	SAMER L5→L3, all pairs (8,310 changed, 6,033 unchanged)	14,343	minimal: literary word swaps
[S1]	Baseet level 3, meaning-filtered	3,508	light
[S2]	Baseet level 2, meaning-filtered	1,954	medium (evaluated here)
[S3]	Baseet level 1, meaning-filtered	1,426	strong
[SA]	DAASI train	1,502	everyday and administrative style

Model-2 training. Both architectures were trained on the same data with one shared script, so that the architecture is the only difference: 10 epochs, batch 8 with 2 gradient-accumulation steps, fp16, maximum length 256, seed 42, evaluation twice per epoch with patience 3, and the checkpoint chosen on SARI over the 600 changed rows of a fixed 900-row SAMER dev sample. AraT5v2 used learning rate 10^{-4} and the prefix :بسط; AraBART used $5 \cdot 10^{-5}$ and :بسط without the shadda, which its tokenizer maps to an unknown token. The script disables length-grouped batching, which in the pinned transformers 4.46 also shuffles the evaluation set and would score each prediction against another row’s reference; it stops if predictions and references ever fall out of order, and the logs show full alignment at all 40 evaluations.

2.3. Workflow

Each task is a GitHub issue with an owner, a reviewer and a definition of done; changes arrive as reviewed pull requests. Over the period reported, the team opened 20 issues and 24 pull requests, of which 20 were merged, with work from every member. SAMER text and model outputs on SAMER move only through private Kaggle datasets and a private Hugging Face organisation, never through the repository.

Figure 2 shows the data path and its three gates. The leakage check compares every training file with both locked test sets, by exact match and by character 5-gram Jaccard overlap of 0.8 or more. The meaning filter admits LLM-written pairs only if they keep the source’s meaning. Generated pairs pass validators before release. The locked test sets are fixed by row count and SHA-256 in a committed manifest and re-verified before every test run.

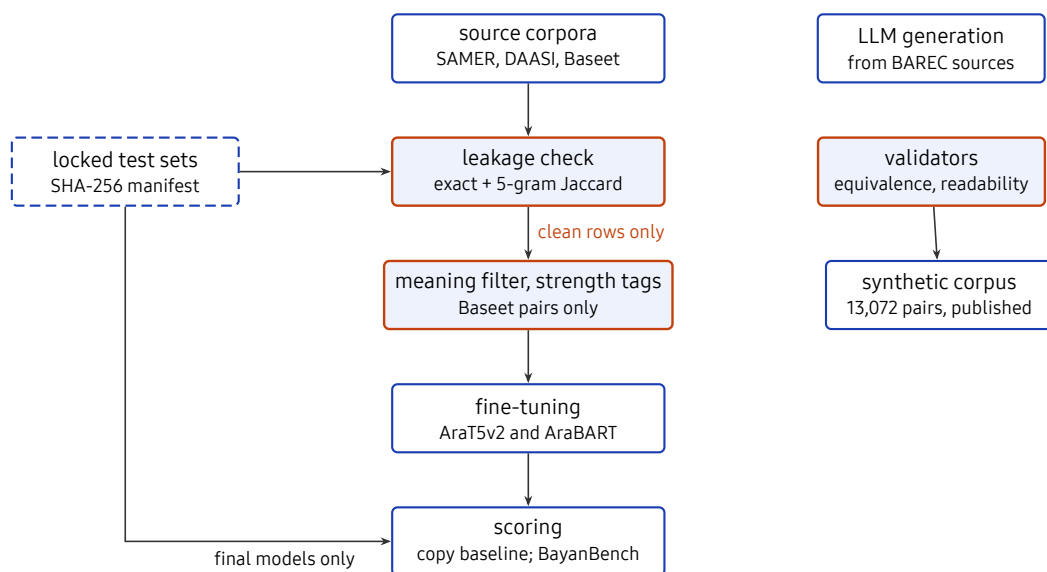


Figure 2. Data and evaluation workflow. Orange boxes are the gates that can reject data. The generated corpus (right) is a separate, published artefact.

2.4. System Design

Figure 3 shows the system as tested on a phone. The Android app registers an activity for `android.intent.action.PROCESS_TEXT` (text/plain, Android 6.0 and later), so تبسيط appears in the selection menu of any app. A translucent activity draws the result over the host app without the overlay permission; an optional floating panel, which does require it, keeps reading while the reader returns to the page.

Inference. The simplifier ships as an int8 ONNX bundle (encoder, decoder with a key-value cache, and tokenizer), downloaded once, verified by SHA-256, and run by ONNX Runtime with a greedy decoding loop written in Kotlin that blocks repeated 3-grams and stops at 256 tokens. The app strips tashkeel, splits the selection into sentences and decodes them one by one, streaming each token to the screen as it is produced. Sentences under six words (headings, labels) pass through unchanged, because the model was trained on longer sentences and invents text around fragments. Each output then passes a *retention fallback*: when a rewrite keeps less than 35% of the source’s words, the source sentence is shown instead. The original is always one tap away.

Read-aloud and diacritization. Voices run on the device through sherpa-onnx. The default,

Nabra-7M-Distill, is a Kokoro-architecture Arabic voice exported to int8 (8.7 MB); three Piper voices are optional downloads. The spoken word is highlighted as it is read. The diacritizer, Libtashkeel, was selected on the benchmark in Table 17 but is not part of the build we tested.

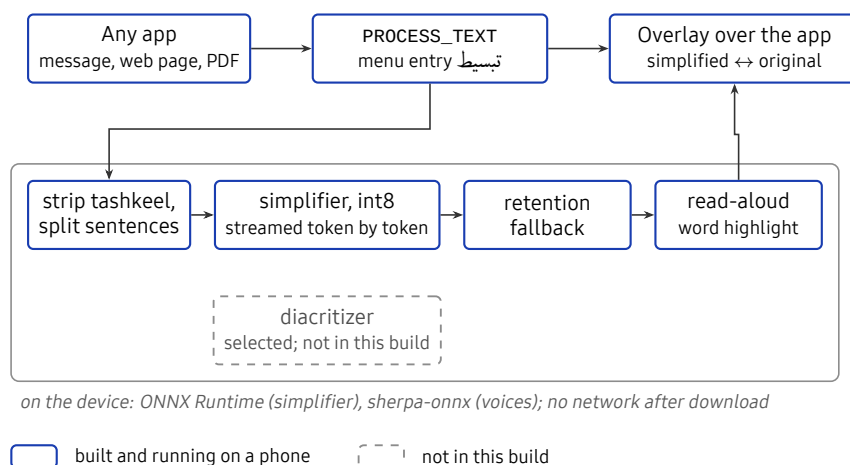


Figure 3. System architecture of the build tested on the phone. The app is entered from the host app’s selection menu and answers with an overlay on top of it.

2.5. BayanBench

SARI measures agreement with one editor’s rewrite. A reader with dyslexia needs something else: that a selection comes back *easier to decode* and *with the same meaning*, and that what should not change does not. No public benchmark tests this for Arabic, so we built one against the construct-validity checklist of [25] and the reporting practices of [26].

Construct. A rewrite is dyslexia-friendly when it says the same thing in a form that is easier to decode and follow: no clause over 15 words (BAREC’s guideline limit for level 10, about grade 4), more familiar words, no new errors, and restraint where the text is already easy. Visual formatting, read-aloud and tashkeel are out of scope, and summarising is always a failure: dropping content never counts as simplification.

Tracks and measures. Each track targets one behaviour and is scored on its own (Table 6); the probe tracks follow CheckList [24]. A *clause* is a maximal span between punctuation marks (full stop, question and exclamation marks, comma, semicolon, colon, line break), so the clause measure credits added commas as well as split sentences and must be read next to the content measures. *Kept words* is the share of the source’s words of three or more letters, compared after removing clitics, that reappear in the output. *Hard words* are words at levels 4–5 of the SAMER readability lexicon, and *reading level* is the maximum sentence level predicted by the CAMEL BAREC readability model, counted only for sources at level 10 or above. Numbers are compared as multisets after mapping Arabic-Indic and Persian digits to Western ones; negations are counted over eleven negating particles.

Items and splits. The 1,981 items form four sets that are reported side by side and never averaged. *Core* items come from BAREC test documents. *Held-out* items are news and legal text, including 247 BAREC sentences of those domains that no model trained on. *Outside* items are 344 sentences from UN News, WHO fact sheets, newly created Arabic Wikipedia articles, UNHCR help pages and Android help, fetched on 28 September and therefore published after every corpus we use. *Written* items are 50 medicine-leaflet sentences written for the benchmark by an LLM, kept out of core because they are not real text. Items are split by document

Table 6. BayanBench tracks, with items per track in the dev / test splits.

Track	Behaviour	Measured by	Items
A1–A3	meaning kept: general; hard spots (negation, numbers, conditions, cause, quantity, certainty); high stakes (legal, medical)	judge (insertion, deletion, substitution; severity [21]); code (numbers, negations)	299 / 508
B	long clauses fixed	code: no clause over 15 words	104 / 187
B2, B3	fewer hard words; lower reading level	SAMER word levels; CAMEL BAREC readability model	across items
C	easy text left unharmed	code: unchanged; judge	129 / 202
D	correct Arabic	judge; house punctuation rules	across items
E1–E4	real selections: cut-offs, messy text, 40+ words, several sentences	code (end mark, clause length); judge	139 / 263
F	protected text (Qur’an, Bible, classical poetry)	code: letters unchanged	60 / 90
G	on the phone	int8 bundle against the same model in PyTorch; size, time	—
H	people	blind ratings on 100 items	—

(731 dev items from 129 documents, 1,250 test items from 197), so no document contributes to both splits. Any BAREC document with a sentence found in any training or generated data was dropped whole (17 documents), and near-copies were removed by word-trigram overlap.

Two tiers. The *code tier* is deterministic: every measure above, plus house punctuation rules, unchanged easy text and protected letters. The *judge tier* holds meaning and grammar verdicts from Gemini 3.1 Pro with a fixed Arabic checklist; the judge sees only the selection and the output, never the system’s name. Verdicts are shared data keyed by the content of the (selection, output) pair, so scoring never calls a judge, an unjudged output is reported as pending rather than guessed, and a judge-tier rate is shown only once 90% of its items have verdicts.

Validating the judge. We planted seven kinds of error (a changed number, a dropped negation, a swapped quantifier, a deleted clause, an invented detail, a mid-sentence full stop, broken agreement) in outputs the judge had passed. It flagged a major error in all 47, named the planted type in 45, and gave the same verdict on 12 of 13 pairs judged twice. Any other judge is accepted only if its verdicts on a 792-pair validation set clear thresholds fixed before any candidate was tested: recall of meaning errors ≥ 0.80 , pass rate on clean pairs ≥ 0.70 , $\kappa \geq 0.60$ against the reference verdicts, planted errors $\geq 90\%$ and recall of language errors ≥ 0.60 . Model 2’s training data contains nothing the judge wrote or approved, and the benchmark’s written items come from a different model family, which limits preference leakage [28].

Statistics and release. Every rate carries a 95% interval from a bootstrap over whole documents (2,000 resamples), because sentences from one document pass or fail together [27]; a comparison counts only when the interval of the paired difference excludes zero. The benchmark ships as a Python package whose command-line tool covers the whole cycle (verify, predict, score, pending, judge, validate-judge) with its data in a checksum-pinned Hugging Face dataset, so any team member can score a model on a laptop, on Kaggle or on Linux.

3. Results

3.1. Data Preprocessing

Conversion. SAMER’s aligned levels were converted to source–target pairs (level 5 → level 3). One converter turns each locked test file into the scoring format, keeps every row (including the 1,599 SAMER test rows, 48.8%, where the two levels are identical) and checks the row count before writing.

Normalisation for comparison. For leakage checks and scoring alignment, diacritics and punctuation are removed and the alef, ؤ, and ي/ي variants are unified. Training text keeps its spelling apart from tashkeel and tatweel.

Leakage. Every training file was checked against both locked test sets and found clean. Matches under five words are reported but not counted, because short formulaic fragments recur in any corpus; BAREC alone repeats 311 sentences between its own train and test splits.

Contamination in a public corpus. 3,190 of Baseet’s 34,294 rows (9.3%) overlapped our locked test sets: 1,829 SAMER test sentences and 1,361 BAREC test sentences. Seen from the other side, 47% of SAMER’s substantive test sentences appear somewhere in Baseet. Training on it unfiltered would have inflated every later score with no visible error. The rows were removed and the remaining 31,104 re-checked: zero matches.

Meaning filter. Applied to the Baseet candidates, the filter keeps 6,888 pairs across the three levels. Its purpose shows at the strongest level: the 5th percentiles of Baseet’s level-1 targets are 0.44 LaBSE similarity and 26% of the source’s length, and 10% of those targets drop a number, against 0.646, 67% and none for DAASI’s human rewrites.

3.2. Exploratory Data Analysis (EDA)

Table 7. Corpus statistics, computed on the files we hold. Identity = target equals source after removing diacritics; length ratio = median of target words over source words.

Corpus	Rows	Median words	≤3 words	Diacritized	Identity	Length ratio
SAMER train	14,343	7	16.4%	0.0%	42.1%	1.00
SAMER test (locked)	3,277	6	23.6%	0.0%	48.8%	1.00
BAREC test (locked)	7,286	10	16.9%	53.8%	—	—
Baseet, strongest level	31,104	14	0.0%	36.5%	0.1%	0.73
DAASI	1,509	12	8.4%	32.3%	0.0%	1.00

Four properties of the corpora (Table 7, Figure 4) explain the behaviour of both models:

- **SAMER carries no diacritics**, while 54% of BAREC test and 36% of Baseet do. A model trained only on SAMER has never seen a diacritized input.
- **SAMER simplifies words, not sentences.** Its median length ratio is exactly 1.00: the editors substituted vocabulary and kept structure. Baseet’s strongest level shortens a 14-word median source to 9 words.

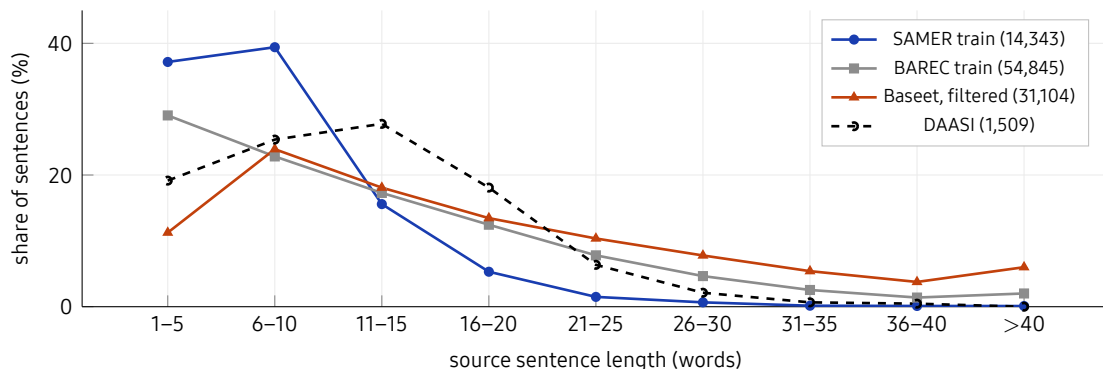


Figure 4. Source sentence length by corpus. SAMER is short literary text; Baseet and BAREC reach the long sentences readers struggle with.

- **No corpus teaches splitting.** A sentence is added in 0% of SAMER pairs, 2–4% of Baseet’s and 8% of DAASI’s, and word length and clitic load barely change in any of them. Existing data can teach shorter, more common words; it cannot teach the restructuring the reading research supports most directly [6].
- **Half of SAMER needs no change.** 42–49% of pairs are identical, and SAMER splits by chapter rather than by novel, so its test set is in-domain. Both make SARI on SAMER read optimistically.

3.3. Modeling

Setup. Reference-based scores use SARI [18] through the `evaluate` implementation, beside BLEU, BERTScore between source and output [19], and the fall in predicted BAREC level. Every score is paired with the copy baseline and split into rows a human changed and rows a human left unchanged. Each final model was run once per test set after training and selection were complete.

RQ1: the reference-based view. Because half of SAMER needs no change, copying scores SARI 77.50 on the full test set but only 56.06 on the 1,678 rows a human changed, and the second number is the bar a model must clear. Table 8 and Figure 5 report model 1, with the harness validated first: it reproduced the training run’s 2,983 dev predictions row for row.

Table 8. Model 1 on test data with human references, scored once with `score.py`. SARI is split by whether the human reference changed the source (DAASI has no unchanged pairs). BERTScore compares output with source; readability drop is the fall in predicted BAREC level (positive = easier). DAASI sources and references have tashkeel stripped.

System	SARI all	SARI changed	SARI un- changed	BLEU	BERT- Score	Read- ability drop	Copy rate
SAMER test (3,277 rows, locked, in domain)							
Copy the input	77.50	56.06	100.00	72.03	1.00	0.00	100.0%
Model 1, greedy	75.88	61.83	90.62	71.68	0.98	0.08	65.3%
Model 1, beam 4	76.29	61.95	91.34	72.22	0.98	0.07	69.4%
DAASI held-out (350 pairs, government and insurance text, out of domain)							
Copy the input	45.05	45.05	—	25.25	1.00	0.00	100.0%
Model 1, greedy	43.87	43.87	—	23.66	0.99	−0.03	77.7%

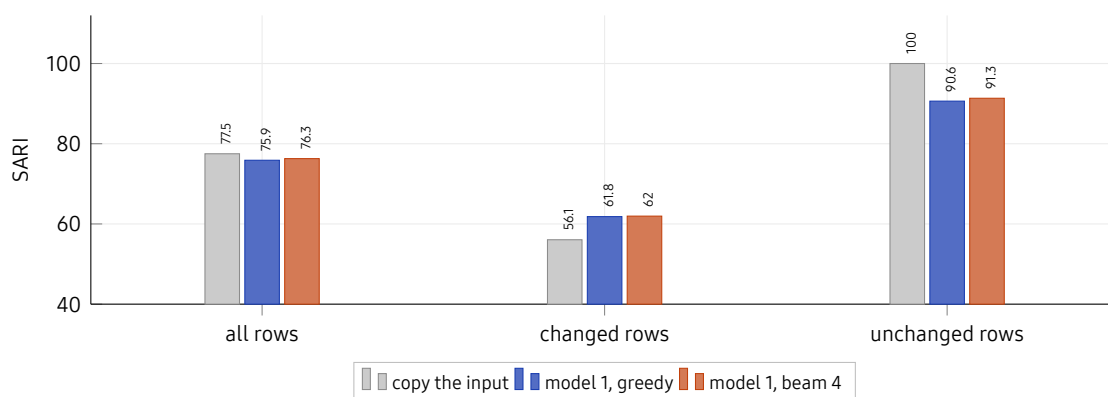


Figure 5. SARI on the locked SAMER test set by reference bucket. Model 1 clears the copy baseline where a human simplified and loses to it where a human left the sentence alone.

Model 1 simplifies where it should and over-edits where it should not. On the 1,678 test rows a human simplified, greedy decoding scores 61.83 against the copy baseline's 56.06 (+5.77 SARI). On the 1,599 rows a human left alone it scores 90.62 against a perfect 100, because it edits 13.6% of sentences that needed no change. The two effects net to -1.62 overall, so the headline number would report a working model as worse than doing nothing. Beam search changes little (61.95 on changed rows).

What model 1 learned. The model returns 65.3% of test rows unchanged where the editors left 48.8%, and leaves 45.3% of the rows an editor rewrote untouched. When it edits, 75.4% of its edits change one word (editors: 49.5%), and its median output-to-input length ratio is 1.00: it substitutes vocabulary and never restructures (Figure 7a). Figure 6 shows the role of checkpoint selection: the SARI that chose the checkpoint had been flat within noise since step 1,800, while the copy rate was still falling.

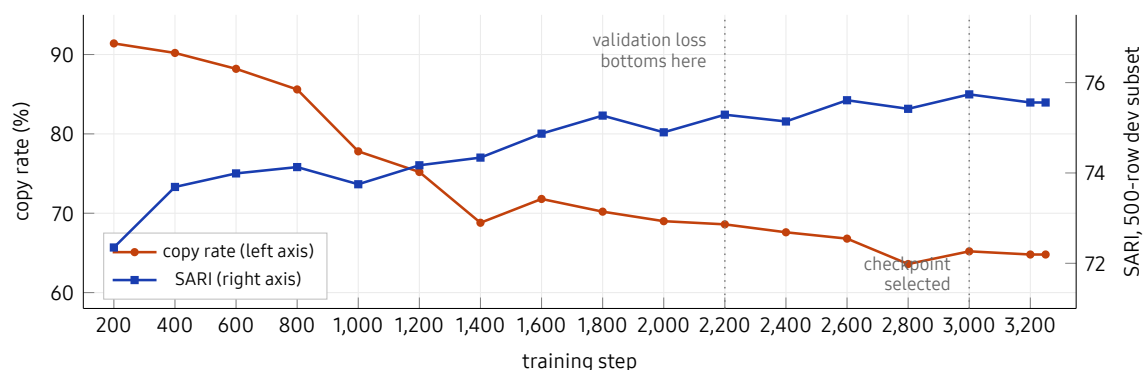


Figure 6. Model 1 training. The checkpoint was selected on a 0.18-point SARI difference while the model's copying behaviour was still changing.

Diacritized and out-of-domain input. On BAREC's 3,918 test sentences that carry tashkeel, raw input keeps 63.7% of the source words and 31.9% of outputs lose more than half the sentence; with tashkeel stripped the same sentences keep 94.6% and 1.8% lose more than half (Table 9, Figure 7b). On DAASI's held-out government and insurance text, model 1 scores 43.87 against a copy baseline of 45.05 (-1.19) and returns 77.7% of sentences unchanged: from SAMER's novels it learned when not to edit, not how to simplify administrative prose.

Meaning errors. On dev, 832 of model 1's substitutions disagree with the editor, 347 of them on words the editor deliberately kept, and some reverse the meaning: عرلة (isolation) became

Table 9. Model 1 on the locked BAREC test set (7,286 sentences, no reference simplifications), greedy decoding. Stripped: tashkeel removed from the source before the model sees it, as the application does.

Input	Copy rate	BERTScore	Readability drop	Source words kept	Rows losing >half
Stripped (as deployed)	68.6%	0.98	-0.04	94.9%	1.4%
Raw, with tashkeel	37.5%	0.91	-0.45	78.2%	17.5%

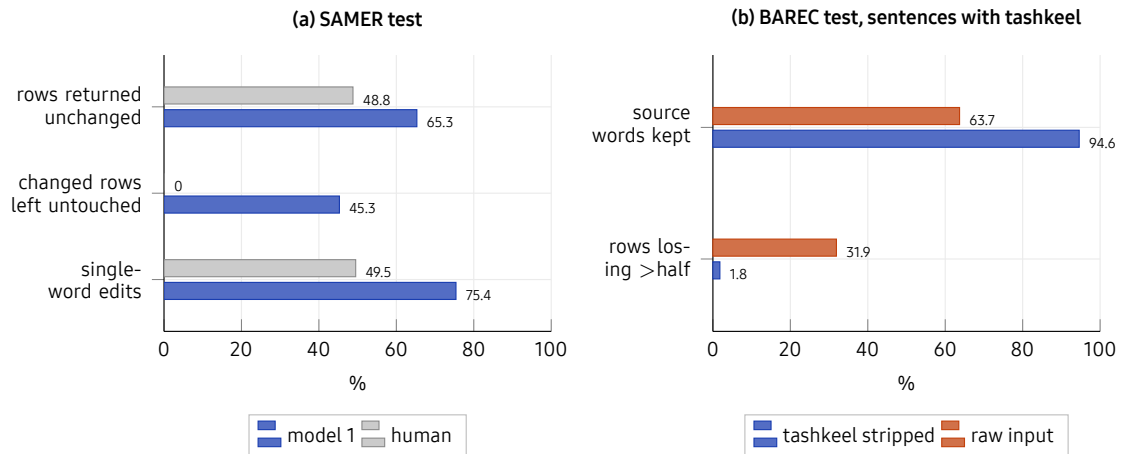


Figure 7. Model 1 behaviour. (a) SAMER test, greedy, against the editors' references. (b) BAREC test, the 3,918 sentences that carry tashkeel, greedy, with and without tashkeel stripped.

استعباد (enslavement), ملجـم (bridled) became مسلـح (armed). SARI and BLEU are blind to this class of error, which is why BayanBench scores meaning separately from ease.

Model-2 training. Both runs clear model 1's selection score (Table 10, Figure 8); for reference, model 1 scored 63.66 SARI on SAMER dev's changed rows (all 1,754 of them, a superset of the 600 used here). AraT5v2 was still improving at its last evaluation; AraBART peaked at epoch 9 and had been flat since epoch 6. Copying falls from 80–91% of rows to about 51% for both, against 33.3% for the editors on the same rows: the recipe reduces over-conservatism but does not remove it, and AraT5v2 still returns 186 of the 600 changed rows unchanged.

Table 10. Model-2 runs, one T4 GPU each. SARI and copy rate on the 900-row selection sample (600 changed, 300 unchanged SAMER dev rows, tag [s0]). Speed: sentences per second on SAMER test sources, fp16, batch 64, greedy.

	AraT5v2	AraBART
Parameters	368M	139M
Training time	6,783 s	2,600 s
Best checkpoint (step of 14,210)	14,200	12,780
SARI, 600 changed rows	66.58	64.71
SARI, all 900 rows	75.64	73.62
Copy rate, 900 rows (editors: 33.3%)	51.0%	51.4%
Generation speed (sentences/s)	80.8 [†]	292.8
[†] measured on model 1, the same architecture and size, in the same run.		

RQ2: model 2 on BayanBench. We ran both models once on the BayanBench test split in the medium setting ([s2]), and also ran the two int8 bundles exactly as the app calls them (Table 11; the dev split, which gives the same picture, is in Table 15). We report the code tier, which is deterministic and complete for every system. Five findings follow.

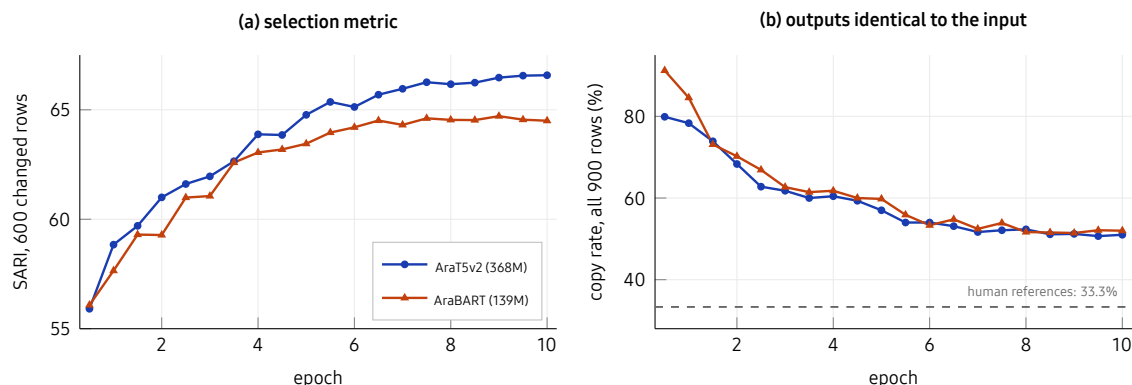


Figure 8. Model-2 training, from the committed notebooks: (a) the selection metric at each of the 20 evaluations; (b) the share of outputs identical to the input, with the editors’ rate as a dashed line.

F1 Model 2 makes text easier on the measures the reading research supports. In the [S2] setting, every clause of a long-clause item ends at 15 words or fewer in 50.8% (AraT5v2) and 55.1% (AraBART) of B items, against 0% for copying; hard words fall in 41.2–44.0% of items that contain any, and the predicted reading level falls in 20.0–21.2%. The two architectures are statistically indistinguishable on every ease measure.

F2 It keeps most of the content, and AraT5v2 keeps more. AraT5v2 keeps at least 60% of the source’s words in 88.2% of items and AraBART in 81.7%, a paired difference whose interval excludes zero. House punctuation rules are broken in about 1% of outputs.

F3 It does not know when to stop. Easy text, which should come back unchanged, does so in only 9.4% (AraT5v2) and 11.4% (AraBART) of C items. Protected text keeps its letters in 4.4%, and a selection cut mid-sentence keeps its ending in about a third of E1 items; the rest are completed with a full stop and, sometimes, invented words (Table 14). No training corpus shows these behaviours: SAMER’s unchanged pairs are literary sentences, and no corpus contains protected or cut-off text.

F4 Numbers and negations are at risk. Numbers survive in 76.3% of items that contain one and negations in 79.3%. Of the 40 number failures, 24 are clause markers such as “(1)” in the Universal Declaration of Human Rights, which a rewrite drops harmlessly; the rest include digits in the Persian range (۱۹۹۲, ۱/۳), which the model drops or replaces.

F5 The deployed input format costs ease. The bundles the app ships send the bare prefix without a strength tag. Table 12 isolates the effect: removing the tag makes the model safer (more words kept, more easy text left alone) and fixes far fewer long clauses. Quantising to int8 changes little for AraBART; for AraT5v2 it moves behaviour further toward copying, making its untagged int8 bundle the most conservative system we measured.

Across item sets. Table 13 reports the two systems a reader would meet, AraT5v2 in the medium setting and the int8 AraBART bundle, on each item set. The ease measures are lower away from core: AraT5v2 fixes long clauses in 58.9% of core items but in 39.1% of held-out and 40.4% of outside items, so part of the core result reflects the genres of BAREC’s documents. The gains nevertheless stay far above copying on text published after every corpus we use. Re-straint is weak everywhere and weakest on held-out and outside text, where fewer than 7% of easy sentences come back unchanged.

Error analysis. Table 14 shows test outputs of AraT5v2 in the medium setting. The first is the intended behaviour: a 17-word sentence split into two cause-linked clauses. The second

Table 11. Model 2 on the BayanBench **test** split (1,250 items, 197 documents), run once, code tier. Pass rates in %, with the 95% interval from a bootstrap over whole documents in brackets. The first two model columns send the [s2] (medium) strength tag the model was trained with; the two *int8 app* columns are the quantised bundles the app downloads, which send no tag. All outputs pass through the app’s retention fallback and punctuation fix. Bold: the best model on each row. *n* = items the measure applies to.

Measure (pass rate, %)	Copy the input	AraT5v2, [s2]	AraBART, [s2]	AraT5v2, int8 app	AraBART, int8 app	<i>n</i>
Easier						
No clause over 15 words (B, long clauses)	0.0	50.8 [44–57]	55.1 [48–62]	18.7 [13–24]	43.9 [38–50]	187
No clause over 15 words (E3, 40+ words)	25.0 [14–36]	46.9 [35–57]	50.0 [39–60]	18.8 [9–29]	34.4 [22–48]	64
Fewer hard words (items with any)	0.0	41.2 [37–45]	44.0 [39–49]	24.3 [21–28]	43.0 [39–47]	588
Lower reading level (sources at level 10+)	0.0	20.0 [17–23]	21.2 [18–24]	8.2 [7–10]	15.3 [13–18]	898
Faithful						
Kept 60%+ of the source’s words	100.0	88.2 [86–90]	81.7 [78–85]	98.4 [98–99]	91.5 [89–94]	958
Numbers kept (items with numbers)	100.0	76.3 [56–94]	74.0 [57–89]	75.7 [55–93]	71.0 [51–88]	169
Negations kept (items with a negation)	100.0	79.3 [74–84]	80.6 [75–86]	92.4 [89–95]	84.4 [80–88]	237
No house-rule break	100.0	99.1 [98–100]	99.0 [98–100]	99.2 [99–100]	99.1 [99–100]	1,160
Restraint						
Easy text left unchanged (C)	100.0	9.4 [6–13]	11.4 [7–16]	33.2 [26–40]	19.8 [14–26]	202
Cut-off selection keeps its ending (E1)	100.0	35.1 [25–46]	37.2 [26–49]	37.2 [28–48]	43.6 [33–55]	94
Protected text: letters unchanged (F)	100.0	4.4 [0–10]	4.4 [0–11]	47.8 [30–70]	24.4 [16–36]	90

passes every code check and reverses a quantity, which is the reason meaning needs a judge tier and not code alone. The others are the failures of F3 and F4. One example is not printed: in a Qur’anic verse (*al-Kāfirūn* 109:4) the model dropped the negation ﻻ, inverting the verse’s meaning, the most serious error we observed and the reason protected text must never reach the model.

Table 12. Paired effects on the test split, in points, with 95% document-bootstrap intervals; * = the interval excludes zero. Same items, same model, one change at a time.

Change	No clause over 15 (B)	Fewer hard words	Kept 60%+ of words	Easy text unchanged (C)
AraT5v2: drop the [s2] tag	-13.4* [-19, -8]	-7.1* [-10, -4]	+7.0* [+5, +9]	+9.9* [+6, +14]
AraBART: drop the [s2] tag	-11.2* [-18, -5]	-2.7 [-6, +1]	+11.5* [+9, +14]	+8.4* [+4, +14]
AraT5v2: PyTorch fp32 → int8 ONNX	-18.7* [-25, -12]	-9.7* [-12, -7]	+3.2* [+2, +5]	+13.9* [+9, +19]
AraBART: PyTorch fp32 → int8 ONNX	+0.0 [-3, +3]	+1.7* [+0, +3]	-1.7* [-3, -0]	+0.0 [-3, +3]

Table 13. Model 2 on the four BayanBench item sets (test split), pass rate in %, number of items in brackets. *Core*: BAREC test documents; *held-out*: news and legal text; *outside*: 344 web sentences published after every corpus we use; *written*: medicine-leaflet sentences. The sets are reported side by side and never averaged.

System and measure	Core	Held-out	Outside	Written
AraT5v2, [s2]				
No clause over 15 words (B)	58.9 (107)	39.1 (23)	40.4 (57)	—
Fewer hard words	44.8 (328)	34.1 (129)	37.4 (123)	62.5 (8)
Lower reading level	22.4 (545)	20.9 (163)	13.2 (167)	8.7 (23)
Kept 60%+ of words	88.3 (582)	86.8 (182)	88.3 (171)	95.7 (23)
Easy text left unchanged	14.3 (77)	6.5 (92)	6.1 (33)	—
AraBART, int8 app				
No clause over 15 words (B)	43.0 (107)	30.4 (23)	50.9 (57)	—
Fewer hard words	45.7 (328)	37.2 (129)	43.1 (123)	25.0 (8)
Lower reading level	16.0 (545)	16.6 (163)	12.6 (167)	8.7 (23)
Kept 60%+ of words	93.5 (582)	89.0 (182)	87.1 (171)	95.7 (23)
Easy text left unchanged	24.7 (77)	18.5 (92)	12.1 (33)	—

Table 14. Test outputs of AraT5v2 ([s2]), from the BayanBench test split. Glosses are ours.

Track	Source	Output	What happened
B	فالمهم الصفح والتسامح وذلك للتخفيف على الصحة النفسية للفرد ولانتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	فالمهم الصفح والتسامح؛ لأن ذلك يساعد على تحسين الصحة النفسية للفرد، ويساعد على انتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	Split into two clauses joined by a stated cause; meaning kept.
B	منذ قرنين على الأكثر بدأ الإنسان يدخل دائرة حلزونية متصاعدة متفاقمة بين الطلب والاستجابة للطلب ...	منذ قرنين على الأقل، بدأ الإنسان يدخل دائرة حلزونية متصاعدة بين الطلب والاستجابة له ...	“At most two centuries” became “at least”: a reversal every code check passes.
C	يعتبر منصب الرئيس شاغرا في أي من الحالات التالية:	يعتبر منصب الرئيس شاغرا في أي حالة:	Easy legal text edited; “in any of the following cases” lost its list.
E1	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال صحة المفاهيم	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال اختبار صحة المفاهيم	A cut-off selection completed with invented words and a full stop.
A2	... بنتيجة (١/٣) في المباراة ...	... بنتيجة (/) في المباراة ...	Persian-range digits dropped.

Table 15. The same measures on the BayanBench **dev** split (731 items, 129 documents), for comparison with Table 11.

Measure (pass rate, %)	Copy the input	AraT5v2, [s2]	AraBART, [s2]	AraT5v2, int8 app	AraBART, int8 app	<i>n</i>
Easier						
No clause over 15 words (B, long clauses)	0.0	56.7 [46–68]	62.5 [52–72]	17.3 [11–24]	45.2 [35–55]	104
No clause over 15 words (E3, 40+ words)	38.7 [21–56]	41.9 [25–60]	48.4 [31–67]	32.3 [16–52]	48.4 [29–69]	31
Fewer hard words (items with any)	0.0	42.9 [36–49]	42.4 [36–49]	27.1 [21–32]	40.6 [35–47]	347
Lower reading level (sources at level 10+)	0.0	18.4 [15–22]	17.4 [14–21]	4.9 [3–7]	14.9 [11–19]	511
Faithful						
Kept 60%+ of the source’s words	100.0	92.8 [91–95]	83.9 [81–87]	99.1 [98–100]	91.1 [89–93]	542
Numbers kept (items with numbers)	100.0	86.9 [77–95]	83.6 [73–92]	88.5 [78–96]	88.5 [78–96]	61
Negations kept (items with a negation)	100.0	86.2 [79–93]	85.3 [78–92]	95.4 [92–99]	90.8 [84–96]	109
No house-rule break	100.0	99.0 [98–100]	99.0 [98–100]	99.3 [99–100]	99.6 [99–100]	671
Restraint						
Easy text left unchanged (C)	100.0	14.0 [8–20]	14.0 [9–19]	37.2 [28–46]	23.3 [17–29]	129
Cut-off selection keeps its ending (E1)	100.0	35.7 [23–47]	42.9 [31–55]	44.6 [33–55]	50.0 [37–62]	56
Protected text: letters unchanged (F)	100.0	11.7 [4–26]	6.7 [0–16]	48.3 [21–91]	26.7 [11–55]	60

3.4. On the Device

RQ3: cost on the phone. Both simplifiers were exported to ONNX with the tokenizer inside the bundle and quantised to int8 (Table 16). The AraBART bundle fits the 250 MB budget; the AraT5v2 bundle does not, because 46% of its weights are two untied 110k-row embedding matrices. We tested the app on a Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM) with the AraT5v2 bundle, the harder of the two to fit. Because output streams, the reader waits for the first token, not the whole sentence: 177–318 ms for a paragraph’s first sentence and 55–85 ms for each later one, on a phone that now sits in the mid-range. On the laptop, with the same decoding loop, AraBART is 1.9 times faster than AraT5v2, so its phone latency should be lower still. Quantisation moves AraBART by less than 2 points on every code measure, while it shifts AraT5v2 toward copying (F5).

Table 16. On-device cost. Laptop: 4 CPU threads, median and 90th percentile per BayanBench test sentence, the app’s own decoding loop. Phone: Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM), from the app’s logs over six runs on a two-sentence paragraph.

	AraBART, int8	AraT5v2, int8
Download (encoder + decoder + tokenizer)	222 MB	471 MB
Laptop CPU, per sentence (median / p90)	0.10 / 0.24 s	0.20 / 0.43 s
Phone: model load, cold	—	2.6–7.3 s
Phone: first token, first sentence	—	177–318 ms
Phone: first token, later sentences	—	55–85 ms
Phone: whole sentence	—	0.48–2.24 s
Read-aloud voice (Nabra-7M-Distill, int8)	8.7 MB	

Diacritization. Four systems were benchmarked on the 743-sentence Tashkeela test fixture (Table 17). Libtashkeel was selected: CATT ties it on accuracy (0.01 points, about six characters in 67,576) but is 16 times larger and carries a non-commercial licence, and CAMEL Tools changed the base letters of 32 of 100 sentences, corrupting the text it was asked to vocalise.

Table 17. Diacritization benchmark. DER = diacritic error rate; base-letter corruption measured on 100 sentences.

System	DER	DER, no case	WER	Base letters changed	Size	Licence
CAMEL Tools MLE [36]	16.57%	13.97%	56.73%	32%	129 MB	MIT/GPL
CATT [37]	8.44%	6.92%	29.40%	0%	77.9 MB	CC BY-NC
Mishkal [38]	12.74%	10.12%	39.33%	0%	4.3 MB	GPLv3
Libtashkeel [39]	8.43%	6.91%	29.57%	0%	4.8 MB	MIT

3.5. User Interface

Figure 9 shows the interface on the phone. Selecting Arabic text in any app offers **تبسيط**; choosing it opens the result over the app, the rewrite appearing sentence by sentence as it is decoded. One toggle switches between *Simplified* and *Original*; below it are read-aloud, copy and share, and the current word is highlighted while Bayan reads. The panel can float over the page or shrink to a bar, so the reader keeps the host app in view while listening. The app follows Android’s Material 3 design rather than a design system of its own, and sets Arabic right to left without justification, because justification in Arabic stretches words and opens uneven gaps. Settings hold the reader’s typeface, size (22 sp by default), line height (1.8) and word spacing,

the one spacing adjustment that carries over to cursive Arabic, together with the background, the model, the voice and its speed, and word highlighting.

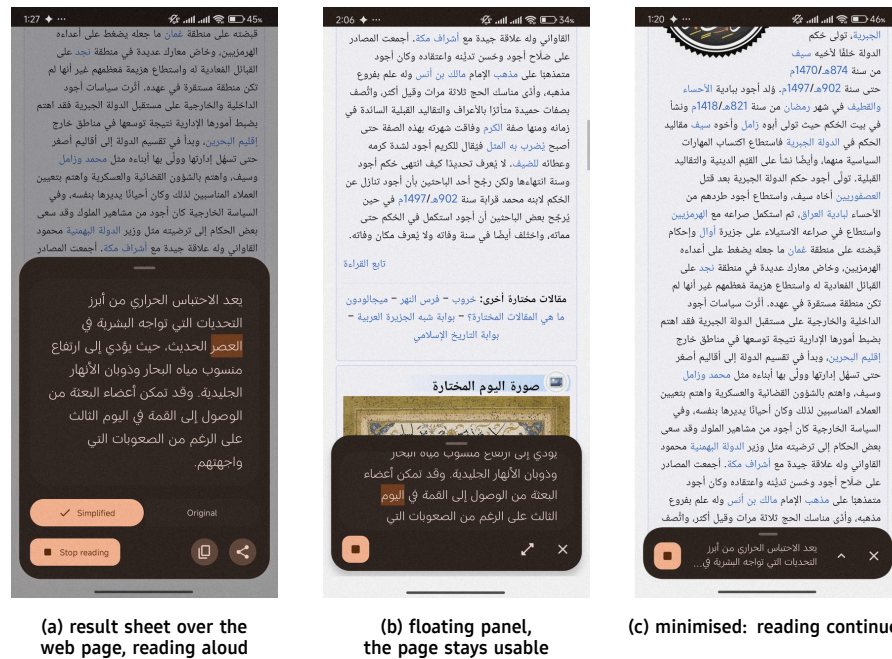


Figure 9. The app on the phone, over an Arabic Wikipedia page. (a) The result sheet, reading aloud with the current word highlighted. (b) The floating panel. (c) Minimised to a bar while reading continues.

3.6. Testing and Improvements

Testing. The first plugin shell shipped 41 unit and Robolectric tests. The app tested on the phone unit-tests its text pipeline (sentence splitting, punctuation repair, clitic normalisation and the retention fallback), and its download pipeline checks every file by SHA-256. The leakage checker is tested against a planted duplicate and must exit non-zero. The benchmark package has its own tests on a built-in toy dataset, and its prediction command reproduces our model-2 outputs on all 731 dev items. Running the pipeline end to end on real data, not fixtures, exposed three harness faults: the scoring script put every BAREC row in the “unchanged” bucket; the shared prediction script omitted model 1’s training prefix, which alone cost twelve SARI points; and length-grouped batching shuffled the evaluation set. Each now has a guard.

Improvements. Each measured problem produced a change (Table 18); the findings of the final evaluation are discussed in subsection 4.2.

Table 18. What the measurements changed.

Finding	Change
Copying scores SARI 77.5 on SAMER test	Copy baseline and changed/unchanged split reported with every score
9.3% of Baseet overlaps our test sets	Rows removed; leakage check required for every training file
Model 1 collapses on diacritized input	Tashkeel stripped before the simplifier, in training and in the app
Model 1 chosen on SARI noise over a copy-heavy dev set	Model 2 selects on changed-row SARI only
Model 1 over-edits unchanged rows yet copies 65% of all rows	All unchanged pairs kept in model-2 training
SAMER teaches word substitution; model 1 below copying on DAASI	DAASI and meaning-filtered Baseet added, each under its own tag
Baseet's LLM targets often summarise	Meaning filter calibrated on human rewrites
Prediction script omitted the training prefix	Prefix a required setting of every model and bundle
Length grouping shuffled the evaluation set	Grouping off; alignment check stops the run
SARI is blind to meaning errors	BayanBench, with a validated judge and code checks
The model invents text around headings and fragments	Sentences under six words pass through unchanged in the app

4. Projected Impact

4.1. Accomplishments and Benefits

Table 19. Objectives from subsection 1.2, against the evidence.

Objective	Evidence	Outcome
O1 Beat copying on changed rows; easier text without losing meaning	Model 1: 61.83 vs 56.06 on SAMER test (+5.77). Model 2: 66.58 on changed dev rows; on BayanBench test, long clauses fixed in 50.8–55.1% of items with $\geq 60\%$ of words kept in 81.7–88.2%; restraint not met	partly met
O2 Diacritize without changing letters	Libtashkeel: DER 6.91% without case endings, 0 of 100 sentences altered, 4.8 MB; not in the tested build	partly met
O3 Run on a phone within 250 MB	AraBART int8: 222 MB; app tested on a Snapdragon 870, first token in 55–318 ms	met
O4 Evidence from readers	Blind rating set of 100 items prepared; no sessions held	not yet

The answers to the research questions are these. **RQ1:** no. On copy-heavy test sets, SARI rewards inaction and is blind to reversals of meaning; a model that beats copying on the sentences editors changed still scores below copying overall, and what it learned from SAMER is vocabulary substitution without restructuring. **RQ2:** largely yes. One tagged model makes long-clause sentences easier in about half of the cases and reduces hard words in over 40%, keeps most content, and still helps, though less, on text published after its training data; it lacks restraint. **RQ3:** yes. The int8 AraBART bundle fits the budget, streamed output reaches the reader in under a third of a second on a mid-range phone, and quantisation costs AraBART little. The shipped bundles are conservative mainly because they send no strength tag, and for AraT5v2 also because of quantisation.

Benefits. For readers: help where and when the difficulty occurs, offline and private, with the

original always one tap away. For schools: a tool that runs on the phones students already own. For the field: a documented case of test-set contamination in a public Arabic simplification corpus, an evaluation protocol that exposes copy-heavy scores, and a benchmark that measures what an Arabic reader with dyslexia needs rather than agreement with one editor.

4.2. Future Improvements

The evaluation points to specific next steps. Restraint and protection belong in the app as much as in the model: scripture and poetry should be detected and never sent to it, a cut-off selection should keep its ending, and Persian-range digits should be mapped before decoding and restored after. The strength tag should travel with every request. Sentence splitting needs training data that teaches it, which no human-written corpus provides. Choosing checkpoints on BayanBench rather than on SARI would let the behaviours readers need shape training itself. Last, the benchmark's human track and a reading study with readers with dyslexia are what would turn these automatic results into evidence about reading.

4.3. Limitations

- **No reader has used Bayan yet.** Every result is automatic. The measures are chosen from reading research, but none is validated as a predictor of reading success for Arabic readers with dyslexia, and no such metric exists in any language.
- **Code tier only.** Meaning is checked here through numbers, negations and kept words. These catch deletions but not substitutions such as the reversal in Table 14, which only the judge tier detects.
- **The clause measure credits commas.** A rewrite can meet the 15-word limit by adding commas as well as by splitting sentences. We read it next to kept words and reading level, and the 15-word limit is itself a design choice: the ranking of the five systems in Table 11 is the same at 12, 15 and 20 words, but the absolute rates move.
- **No reference-based test score for model 2.** Model 2 is compared with published work only through its dev selection score; its test evaluation is BayanBench, which has no human references.
- **Some tracks are small.** E3 and E4 hold 64 and 40 test items and separate only large differences.
- **One phone and one bundle.** On-device figures come from one mid-range phone and the AraT5v2 bundle.
- **One run per configuration.** Each model was trained once, with one seed, so the comparison between architectures carries no estimate of training variance.

4.4. Conclusion

Bayan shows that an Arabic reading aid for readers with dyslexia can run entirely on a mid-range phone, from inside any app, at interactive latency. Getting there depended less on model size than on evaluation. The field's standard score hid both what a model had learned and what it broke. A public corpus overlapped the test sets. And the behaviours a reader needs — shorter clauses, familiar words, the same meaning, and restraint where the text is already easy — had to be measured one by one before they could be improved. BayanBench makes those measurements reproducible by anyone on the team, and its first results mark restraint as the problem to solve next.

References

- [1] A. F. Aldakhil. Prevalence of developmental dyslexia among primary school children in Arab countries: a systematic review and meta-analysis. *Research in Developmental Disabilities*, 152:104812, 2024. doi:10.1016/j.ridd.2024.104812
- [2] E. Saiegh-Haddad and H. Taha. The role of morphological and phonological awareness in the early development of word spelling and reading in typically developing and disabled Arabic readers. *Dyslexia*, 23(4):345–371, 2017. doi:10.1002/dys.1572
- [3] L. Rello, R. Baeza-Yates, L. Dempere-Marco and H. Saggion. Frequent words improve readability and short words improve understandability for people with dyslexia. *INTERACT*, pp. 203–219, 2013. doi:10.1007/978-3-642-40498-6_15
- [4] M. Rivero-Contreras, P. E. Engelhardt and D. Saldaña. An experimental eye-tracking study of text adaptation for readers with dyslexia: effects of visual support and word frequency. *Annals of Dyslexia*, 71(1):170–187, 2021. doi:10.1007/s11881-021-00217-1
- [5] I. A. Asadi, A. Asli-Badarneh and V. Vaknin-Nusbaum. The impact of morphological density on reading comprehension in Arabic: a comparison between typically developing and children with reading disabilities. *Dyslexia*, 30(1):e1761, 2024. doi:10.1002/dys.1761
- [6] L. Javourey-Drevet, S. Dufau, T. François, N. Gala, J. Ginestíe and J. C. Ziegler. Simplification of literary and scientific texts to improve reading fluency and comprehension in beginning readers of French. *Applied Psycholinguistics*, 43(2):485–512, 2022. doi:10.1017/S014271642100062X
- [7] E. W. Hermena, D. Drieghe, S. Hellmuth and S. P. Livesedge. Processing of Arabic diacritical marks: phonological–syntactic disambiguation of homographic verbs and visual crowding effects. *JEP: Human Perception and Performance*, 41(2):494–507, 2015. doi:10.1037/xhp0000032
- [8] A. Madjidi and C. Crick. Enhancing textual accessibility for readers with dyslexia through transfer learning. *ASSETS*, pp. 1–5, 2023. doi:10.1145/3597638.3614473
- [9] M. Gharaibeh and Z. Basulayyim. ChatGPT enhances reading comprehension for children with dyslexia in Arabic language. *Disability and Rehabilitation: Assistive Technology*, 20(8):2858–2871, 2025. doi:10.1080/17483107.2025.2508389
- [10] B. Alhafni, R. Hazim, J. D. Piñeros Liberato, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, pp. 16079–16093, 2024.
- [11] K. Elmadani, N. Habash and H. Taha. A Large and Balanced Corpus for Fine-grained Arabic Readability Assessment. *Findings of ACL*, pp. 16376–16400, 2025.
- [12] Baseet: multi-level Arabic text simplification with Aya-8B-generated targets and AraT5v2/AraBART baselines. Graduation project, University of Jeddah, 2026. <https://github.com/raya-y/Baseet>
- [13] DAASI: parallel Arabic simplification of government and insurance text, GPT-4o drafts verified by two annotators. tabsIT project, CCO. <https://github.com/Huseen1141/tabsit>
- [14] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017.
- [15] C. Scarton and L. Specia. Learning simplifications for specific target audiences. *ACL*, pp. 712–718, 2018. doi:10.18653/v1/P18-2113
- [16] L. Martin, É. de la Clergerie, B. Sagot and A. Bordes. Controllable sentence simplification. *LREC*, 2020.
- [17] L. Martin, A. Fan, É. de la Clergerie, A. Bordes and B. Sagot. MUSS: Multilingual unsupervised sentence simplification by mining paraphrases. *LREC*, pp. 1651–1664, 2022.
- [18] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing statistical machine translation for text simplification. *TACL*, 4:401–415, 2016.
- [19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi. BERTScore: Evaluating Text Generation with BERT. *ICLR*, 2020.
- [20] F. Alva-Manchego, C. Scarton and L. Specia. The (un)suitability of automatic evaluation metrics for text simplification. *Computational Linguistics*, 47(4):861–889, 2021. doi:10.1162/coli_a_00418
- [21] A. Devaraj, W. Sheffield, B. C. Wallace and J. J. Li. Evaluating factuality in text simplification. *ACL*, pp. 7331–7345, 2022. doi:10.18653/v1/2022.acl-long.506
- [22] L. Cripwell, J. Legrand and C. Gardent. Simplicity level estimate (SLE): a learned reference-less metric for sentence simplification. *EMNLP*, pp. 12053–12059, 2023. doi:10.18653/v1/2023.emnlp-main.739

- [23] D. Heineman, Y. Dou, M. Maddela and W. Xu. Dancing between success and failure: edit-level simplification evaluation using SALSA. *EMNLP*, pp. 3466–3495, 2023. doi:10.18653/v1/2023.emnlp-main.211
- [24] M. T. Ribeiro, T. Wu, C. Guestrin and S. Singh. Beyond accuracy: behavioral testing of NLP models with CheckList. *ACL*, pp. 4902–4912, 2020.
- [25] A. M. Bean et al. Measuring what matters: construct validity in large language model benchmarks. *NeurIPS Datasets and Benchmarks*, 2025.
- [26] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy and M. J. Kochenderfer. BetterBench: assessing AI benchmarks, uncovering issues, and establishing best practices. *NeurIPS Datasets and Benchmarks*, 2024.
- [27] E. Miller. Adding error bars to evals: a statistical approach to language model evaluations. arXiv:2411.00640, 2024.
- [28] D. Li et al. Preference leakage: a contamination problem in LLM-as-a-judge. *ICLR*, 2026.
- [29] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, pp. 628–647, 2022.
- [30] A. Elmadany, E. M. B. Nagoudi and M. Abdul-Mageed. Octopus: A Multitask Model and Toolkit for Arabic Natural Language Generation. *ArabicNLP*, pp. 232–243, 2023.
- [31] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, pp. 31–42, 2022.
- [32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [33] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *ACL*, pp. 7871–7880, 2020.
- [34] D. Samuel, A. Kutuzov, S. Touileb, E. Velldal, L. Øvrelid, E. Rønningstad, E. Sigdel and A. Palatkina. Nor-Bench – A Benchmark for Norwegian Language Models. *NoDaLiDa*, pp. 618–633, 2023.
- [35] L. Burchell et al. An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT). *ACL*, pp. 17452–17485, 2025.
- [36] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, pp. 7022–7032, 2020.
- [37] abjadai. CATT: Character-based Arabic Tashkeel Transformer. <https://github.com/abjadai/catt>
- [38] T. Zerrouki. Mishkal: Arabic text vocalization. <https://github.com/linuxscout/mishkal>
- [39] Libtashkeel, an ONNX Arabic diacritization model (MIT), via the text2tashkeel wrapper.

5. Team Member Review and Comment

◁ATTACH A TEAM PICTURE HERE▷

NAME	REVIEW and COMMENT
Marwan Elamami	
Abdulrahman Khengari	
Ahmed Alaeb	
Sanad Ali	
Mohammed Thabet	
Abdul Majid Mraied	

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	